Crowd Level Detection in Buses using Camera-Assisted YOLOv8

Dharan C
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology and Advanced Studies, Chennai, India

Anto X
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology and Advanced Studies, Chennai, India

Varun S
Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology and Advanced Studies, Chennai, India

S. Divya Bairavi, M.E., (Ph.D)
Assistant Professor (Guide), Department of Computer Science and Engineering (AI & ML)
Vels Institute of Science, Technology and Advanced Studies, Chennai, India

***Abstract*—Average bus ridership is accustomed to a bus load factor only at the bus stop and there is no efficient way of detecting the same in public transport systems. This paper focuses on the crowd level detection system of buses using the camera-assisted YOLOv8. Camera fitted in the passenger cabin of the bus takes the picture at the bus stop. Crowd level detection algorithm uses the YOLOv8 nano model along with the Raspberry Pi Zero 2 W, and crowd level classification is done according to the number of people, after which the results from the Flask web service are sent to the live dashboard accessible by the passengers and public transport organizations. Only 1 picture is taken at the bus stop instead of videos, and the computational task remains within an acceptable range that can be achieved using the proposed hardware whose approximate cost is ₹2,700 per bus.**

The functional prototype for the proposed solution was created for the laptop webcam. This is done by applying pre-trained weights of YOLOv8 nano, counting the number of people, detecting the crowd level, providing live stream, the number of people and crowd level using the web dashboard, which uses the Flask framework. Further steps in order to implement the solution include Raspberry Pi implementation, thresholding level adjustment for the images of buses, and evaluation of the solution. The number of people and the crowd level can be viewed as an approximate result obtained by the model, not the actual measurement. The architecture of the proposed system, process flow, privacy and security issues related to the solution, the evaluation of the solution, the implemented step, the limitations of the approach, and further steps of the research are analyzed in the paper. Accuracy and latencies of the system are not measured.

***Index Terms*—crowd detection, passenger counting, YOLOv8, edge computing, Raspberry Pi, intelligent transportation systems, Flask dashboard, privacy.**

I. Introduction

Using the bus as means of transportation is common in most places around the world. Crowdedness of a bus affects the level of services provided. A passenger waiting for a bus from the bus stop cannot predict the crowdedness of the bus. When the number of passengers is known, it can either imply that the passenger will have to wait for the next bus or join the already overcrowded one. For transit operators to estimate the crowdedness of different buses during a certain time and place, it is hard because route planning of the buses is done manually.

There are three major requirements in this category. One is the requirement for the passenger to find the level of crowds of the bus at his next stop. Second is the requirement of the transit operator to know how the crowdedness of the bus is affected by the bus stops, bus routes and the time of the day. Lastly, there is the requirement for the evidence for bus planning and scheduling through doing a single low cost measure.There will be no necessity to build any system if it is possible to satisfy the needs of all three users.

There are some limitations of the current methods of counting people. Manual counting and simple sensors may be lacking accuracy or too laborious. Hardware-based APC is very costly and it is impossible to use it everywhere in buses. Wi-Fi and infrared sensors are capable of counting the number of people in the bus but they do not give enough information about passenger distribution in the bus and complex density estimation models cannot be used on cheap edge devices.

In our research, we propose to use one camera in a bus and lightweight object detector (YOLO) to estimate the number of people at bus stops and send this information to the real-time dashboard. There are three main objectives of our approach. Firstly, the cost of the detector should be low enough for a bus company to install these detectors on all their buses and not only on those who participate in the pilot project. Secondly, the detection process should be carried out using the same device as the detector.The output should be visualization that can be independently visualized by the commuter irrespective of the output visualization done by the bus driver.

Contributions of this paper are:

- Design of the system with stopping trigger for the picture capturing, passenger detection using YOLOv8 nano on the edge device, classification of crowd level (including the thresholds), Flask API and the live dashboard.

- Cost estimate of approximately ₹2,700 per bus for the above edge device.
- Prototype of the proposed detection, crowd level classification and dashboard.
  Prototype includes the use of laptop camera.
- Privacy issue, security issues and the analysis of the system design. Quantitative performance analysis is yet to come.

II. Related Work

A. Lightweight Detection and Density Estimation

Ahmed et al. proposed crowd monitoring IoT system using SSD detector and transfer learning approaches [5]. Wang et al. proposed edge intelligence and lightweight CNN for crowd density estimation [6]. Khan et al. proposed lightweight real-time video surveillance density estimation algorithm called LCDnet [7]. Lightweight detectors like MobileNetV2 [3] have characteristics of the embedded detection device.

Zhang et al. developed methods to estimate and map the density of a crowd from video captured from a surveillance camera and the corresponding geographical data using GIS technologies [8]. The approach of this work is to reduce the detector output (people count) to a coarse classification, the low, medium and high crowd-levels that can be read by a passenger at a glance. The proposed framework classifies the detected people count into the low, medium, and high crowd-levels that can be read at a glance by a passenger waiting for a bus. This is in line with the needs of the public and the passengers in a real world setting.

B.Monitoring crowd using YOLO and IoT

The YOLO object detector [1] is known as one-shot detectors, which are extensively used in person detection [1],[2]. YOLO-GHOST entails the monitoring of crowd using IoT, where the frames captured by IP cameras installed in the bus will be transmitted over the internet to the server, where the counting and detection of people will take place before sending the results to the transit authority [10]. They utilized the MATLAB simulation tool in testing their systems [10]. The YOLO-GHOST is the closest system compared to the system being designed in this paper. However, there are differences between the proposed framework and previous systems, which are three. Firstly, the proposed framework will apply stop-triggered photography while the YOLO-GHOST applies video capture. Secondly, the computation will occur in the single board computer at low cost in the bus; thus, there will be no transmission of the pictures from the bus. Thirdly, the proposed framework will have a dashboard for the passengers on the bus.PublicVision was proposed by Qaraqe et al., which is a secure surveillance system for crowd monitoring [9]. The motivation of the research shows why the system needs to secure the backend part, which affects the development of the API of our proposal.

C. Public Transport Occupancy Systems

It is possible to determine the level of occupancy of a bus using IR and WiFi sensor technologies. However, very little information can be acquired regarding the distribution of people in a bus, which can be achieved using vision techniques.

D. Research Gap

From the systems discussed above, it can be seen that the YOLO based detection system and the IoT dashboard and other systems have been developed, which also work according to their purpose. However, all the research work ends up with providing the level of detection accuracy of the number of people detected and no information about the status of crowds is provided from the point of view of the passenger and the heavy density estimation algorithm cannot be installed on low-cost edge devices.

Table I. Related work overview

Year	Author(s)	Methodology & relevancy
2021	Ahmed et al. [5]	SSD with transfer learning for IoT crowd monitoring; a case of detector-based IoT.
2022	Wang et al. [6]	Crowd density estimation in edge-intelligence through lightweight CNN; easy model deployment in edge devices.
2023	Khan et al. [7]	LCDnet, lightweight density estimation in real-time video surveillance.
2023	Zhang et al. [8]	Density estimation and mapping from surveillance video and GIS.
2024	Qaraque et al. [9]	PublicVision, a secure smart surveillance system for crowd behavior; inspires backend security requirement.
2024	Arun et al. [10]	YOLO-GHOST, IP-camera frames in server-side processing (MATLAB simulated); closes system.

III. Crowd Monitoring System Framework Proposal

Crowd monitoring system framework can include five blocks – triggering photo taking at the stops, detection and counting passengers, crowd level classification, backend system which includes a storage facility and live dashboard. Each bus will have one cheap edge device. The device will take photograph at each bus stop, determine people count through a detector from the photograph, classify the crowd levels for the obtained passenger count, push the data to the backend system which gives the information

Figure 1. Proposed pipeline with stop-triggered crowd detection: stop trigger (IR or ultrasonic sensor detects bus arrival at a stop) -> photo capture (camera inside the bus, passenger compartment of the bus) -> person detection and counting (YOLOv8 nano on Raspberry Pi Zero 2 W) -> crowd-level classification (Low / Medium / High thresholds) -> Flask API and database (verified and stored record) -> live dashboard (bus passengers and transit authorities).

A. Edge unit

The edge unit does the photo capture and inference and does not store any data. The edge unit receives the stop trigger,

captures a photo, runs a detector on the photo, and produces a people count and a crowd-level. As the inference is performed locally, no photo leaves the bus. Only the generated record is sent.

B. Backend and dashboard

The rest of the blocks form the backend. The Flask API receives the record from the edge unit and validates the record. Validated records are stored in the SQL database. The web dashboard queries the database for the latest record and updates the crowd status that is available both to the passengers and transit authorities. The capture, detection, classification, API, and dashboard are separate modules and operate

IV. Hardware And Software

A. Hardware Needed

Table II depicts the hardware required for the proposed edge unit. Inference is carried out on-device using Raspberry Pi Zero 2 W. A OV2640 camera or a USB camera (cheap) captures an image, and either IR or ultrasonic sensor triggers the image capturing process at each bus stop. The Raspberry Pi Zero 2 W runs on 5 V supplied by the 12V to 5V buck converter running from the 12V power line of the bus. The edge unit is enclosed in a 3D printed/acrylic case (Fig. 2). The estimated total cost is about ₹2,700 per bus. Dedicated APC is many times expensive as compared to that, and hardware also comes at a greater cost.

Fig. 2. Power and Signal Chain of the proposed edge unit: Bus 12V power line → 12V to 5V buck converter → Raspberry Pi Zero 2 W (On device inference) → Camera and stop trigger sensor (Input) → Enclosure (3D printed / Acrylic)

Table II. Hardware cost per bus

Component	Function	Cost (₹)
Camera Module (OV2640/USB)	Image capturing	350
Raspberry Pi Zero 2 W	Inference processing	1,500
IR/ultrasonic sensor	Stop sensor	150
MicroSD card, 16 GB class 10	OS, weights, images	300
12V to 5V buck converter	Power supply on bus line	150
Case	Protection of camera and Pi	250
Total		2,700

B. Software specification

The detector used in the project is the YOLOv8 nano detector provided by Ultralytics [2]. This was selected due to its lightweight feature and high speed, which makes it suitable for running on the low-cost device. YOLOv5 nano can be an alternative detector for this project. The software has been written in Python. OpenCV [4] will be used for image capturing and pre-processing. Web server programming will use a language that supports the Flask framework to create RESTful web services. YOLOv8 nano people detection records will be stored in either the MySQL or SQLite database. Raspberry Pi OS will be the edge device's operating system.

V. Methodology

Crowd monitoring pipeline includes six stages.

1. Trigger. The trigger indicating that the bus has stopped at the bus stop occurs based on the IR or ultrasonic sensor determining that the bus has stopped.
2. Capture. The in-bus camera captures the image of the interior of the vehicle.
3. Detection. People in the image are detected and counted using the YOLOv8 nano detector on the Raspberry Pi.
4. Classification. The count of people is classified into low, medium, and high crowd-level depending on the thresholds.
5. Storage. The record is checked and saved in the SQL database using the Flask API.
6. Update. The dashboard queries the latest record and updates the crowdstatus of the passengers and transit authorities.

Given the way the image is captured at each bus stop, it will guarantee that the inference task for the image will be fewer than that for video. There will be a huge shift in crowd-level when the passengers are boarding and alighting from the bus; hence, the number of people at each bus stop is expected to be current. This is among our expectations of the evaluation plan and still to be done.

VI. Classification Of Crowd Level

The number of passengers inside a bus is detected and classified according to the thresholds provided in table III. For example, a passenger count of 25 indicates that the bus is categorized as a medium level of crowd. The threshold values are based on the assumption of a medium sized bus operating in the city. It is assumed that the bus has a capacity of 40 seats but it can accommodate up to 70 passengers. A low level of crowd refers to an empty seat in the bus while a medium level of crowd refers to a full seat with extra space. The high level of crowd refers to a fully occupied bus.

These are the initial values of the threshold levels. This is an absolute number of passengers rather than percentage of capacity of the bus. This means that the threshold values cannot be applied to all sizes of buses. Furthermore, the threshold values need to be adjusted according to the environment of bus operations.

The crowd-level rule can also be expressed in the normalized form in terms of capacity $c = N/C$ where C is the capacity of the bus which equals about 70. Low crowd-level if $c < 0.21$, medium crowd-level if $c < 0.57$, and high crowd-level if $c > 0.57$. All other buses can also use the crowd-level rule but should modify the capacity of buses. Crowd-level is just an estimation depending on the passenger count and cannot be considered as an exact calculation of the population in the bus.

Table III. Crowd-level thresholds

Crowd level	Passenger count	Example
Low	0-15	Count 8 → Low
Medium	16-40	Count 25 → Medium
High	More than 40	Count 52 → High

VII. Implementation

A. Person detection and counting

In the present prototype, the weights of yolov8n model are pre-trained, and the detector is invoked for each frame. Only persons classes detectors are taken into account.A bounding box encloses each detection, and total number of bounding boxes determines the number of people inside the bus. The script that performs people detection and counting has been implemented in a different file called count_people_yolo.py. Test_yolo.py file has been used to test the people detection on static images.

B. Integration of live camera

The OpenCV frame capture reader captures the frame from camera. At present, the prototype implementation works with the camera being the laptop webcam, and the tests were carried out with the laptop webcam. The prototype implementation has not been tested with the in-bus camera yet. The continuous capturing of the frames and passing them to the YOLOv8 nano detector takes place. The output video is the annotated video. For the web application that is developed on the Flask framework, the capturing occurs on the background daemon thread which ensures the web server's always being responsive. The threaded mode is utilized by the web server.

Fig. 3. Prototype implementation flow with the laptop webcam: camera frames (OpenCV capture, background daemon thread) → YOLOv8 nano (yolov8n, person class only) → people count and crowd level (threshold-based classification) → Flask routes (/, /video_feed, /status) → browser dashboard (annotated feed, count, level).

C. Flask web dashboard

The web dashboard has been implemented using three routes of the Flask web framework. The / route is used to access the dashboard web page, the /video_feed route will give a live stream of the annotated camera feed, and the /status route will return the number of people in the bus and crowd level. The numbers of people and crowd level will be periodically updated on the web page.

VIII. Algorithms To Be Proposed

A. Stop triggered counting on the edge unit

1. Wait until stop trigger is received from the IR or ultrasonic sensor.
2. Take a single image of the passenger compartment.
3. Apply the YOLOv8 nano detector to the captured image. Filter out only the detection of the person class.
4. The number of detected objects is the number of passengers in the bus.
5. The number of passengers is converted to low, medium, and high crowd levels based on the threshold.
6. Pass the number of people and crowd level (record) to the Flask API.
7. Delete the captured image after counting.

B. Update the back end and refresh the dashboard

1. Authenticate the record via the API.
2. Validate the record and store in SQL database.
3. Publish the latest record via the status route.
4. The dashboard will refresh itself (passengers and transit authorities).

IX. Security And Privacy

The system records the images of the passengers who board the bus; hence, privacy becomes one more issue to consider in this system. Four layers of security are introduced in order to keep the passengers' privacy.

- The image processing is carried out at the edge node; hence, no image is sent to the cloud-based service.
- It publishes only the number of people and congestion levels inside the bus.
- The image is deleted once the counting process is over.
- API authentication.

It is based on the previous research conducted regarding privacy of surveillance systems [9]. These are some key requirements in designing such a system. The privacy aspect is not studied yet.Figure 4. Architecture of the system and data flow: passenger compartment of the bus → edge unit (capturing, detecting and classifying) → Flask API and SQL database → real-time dashboard → passengers and transit authorities.

X. Results And Analysis

At this point, the results are provided by the prototype only. YOLOv8 nano model has been successfully loaded and used. Person detection and counting are actually working, webcam stream and Flask dashboard that show the number of people and crowd level have been verified. Speaking of dashboard verification with group picture, the detector recognized seven persons, the number belongs to Low band in Table III (in this particular case of dashboard verification different thresholds were used, but they occurred previously). The results of in-bus photos detections are not collected yet.

It is a demo version, and it confirms the possibility of pipeline execution, but no information about its performance, latency and robustness is provided. Moreover, it was tested on the laptop, not on the target device or the bus. All the numerical results that have not been observed have been omitted deliberately. Table IV describes the metrics that should be reported instead.

Table IV. Suggested reporting format

Component	Metric	Value
Person detection	Precision / recall / mAP@0.5	To be measured
Passenger counting	MAE / RMSE	To be measured
Crowd classification	Accuracy / confusion matrix	To be measured
Edge inference	Latency / memory / temperature	To be measured
End to end	Trigger-to-dashboard delay	To be measured

XI. Discussion

It is important to highlight the main trade-off between continuous monitoring and cost cutting. Single measurement per each stop will give enough information to the commuter in advance and will guarantee that the additional processing involved in the system should not be higher than the worth of the board costing approximately Rs 1,500.In other words: The division between the process of capture, detection , verifying, on one hand, and API and dashboard, on the other hand, provides the possibility to change the detector or its configuration in any way without changing anything else in the system.

Counting is just an estimation, and crowd level classification is a coarse one. Single measurement per stop will give no information about passenger distribution inside the bus.

XII. Limitations And Future Work

- The system was validated by implementing the trained model to the laptop webcam. The future steps include the implementation of the system to the Raspberry Pi Zero 2 W board and the performance and inference time for that board are still unknown.
- Pre-trained person detection algorithm can fail to detect some people in highly dense and crowded environments and it could cause the high threshold to be set as medium. The model will be finetuned on the bus images.
- The Low, Medium and High thresholds are the number of people chosen for an average bus